

Popular Machine Learning Methods: Idea, Practice and Math

Recurrent Neural Networks

Yuxiao Huang

Data Science, Columbian College of Arts & Sciences
George Washington University

Fall 2026

Reference

- This set of slides was largely built on the following 7 wonderful books and a wide range of fabulous papers:

HML Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)

PML Python Machine Learning (3rd Edition)

ESL The Elements of Statistical Learning (2nd Edition)

PRML Pattern Recognition and Machine Learning

NND Neural Network Design (2nd Edition)

LFD Learning From Data

RL Reinforcement Learning: An Introduction (2nd Edition)

- For most materials covered in the slides, we will specify their corresponding books and papers for further reference.

Case Study

- See related case study in github repository:
[/p3_c2_s4_recurrent_neural_networks/case_study](#)

Table of Contents

- 1 Learning Objectives
- 2 Motivating Example
- 3 Short-Term RNNs
- 4 Long-Term RNNs
- 5 Encoder-Decoder Networks
- 6 Bidirectional RNNs
- 7 Attention Mechanisms
- 8 Transformer
- 9 Building and Training RNNs
- 10 Bibliography

Learning Objectives: Expectation

- It is **expected** to understand
 - the architecture and idea of short-term Recurrent Neural Networks (RNNs)
 - the architecture and idea of long-term RNNs:
 - Long Short-Term Memory (LSTM)
 - Gated Recurrent Unit (GRU)
 - the architecture and idea of Encoder-Decoder Networks
 - the architecture and idea of Bidirectional RNNs
 - the architecture and idea of Attention Mechanisms
 - the architecture and idea of Transformer
 - the good practices for building RNNs
 - the idea of and good practices for transfer learning using state-of-the-art pretrained RNNs

Learning Objectives: Recommendation

- It is **recommended** to understand
 - the backpropagation for RNNs:
 - Backpropagation Through Time (BPTT)
 - Real-Time Recurrent Learning (RTRL)

IMDB 50K Movie Review Dataset

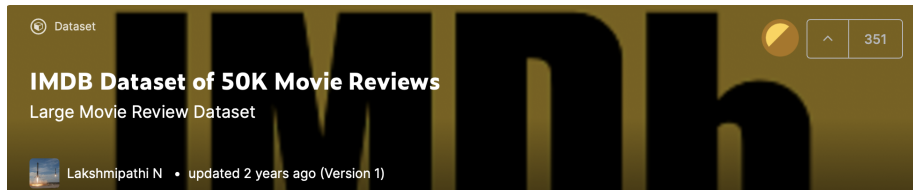


Figure 1: Kaggle competition: IMDB dataset of 50K movie reviews. Picture courtesy of Kaggle.

- [IMDB 50K movie review dataset](#): a large movie review dataset for binary sentiment classification:
 - features: movie review (in text)
 - target: sentiment for the movie (positive or negative)

Recurrent Perceptron



Figure 2: A recurrent perceptron. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- In *Feedforward Neural Networks*, information always passes forward (from lower layer to higher layer).
- In *Recurrent Neural Networks* (RNNs), information also passes backward (from higher layer to lower layer).
- Fig. 2 shows a recurrent perceptron where the output of the output layer goes back to the input of the input layer.

Unrolling Recurrent Perceptron Through Time

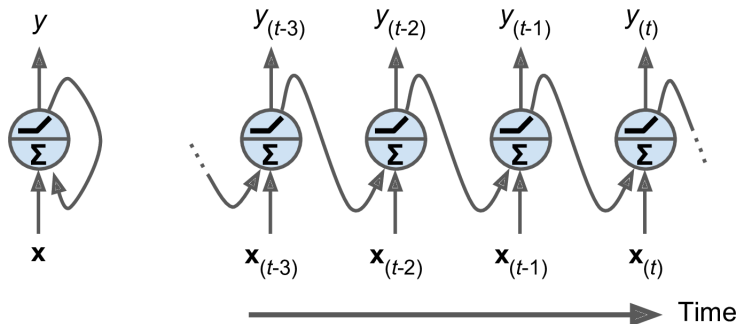


Figure 3: A recurrent perceptron unrolled through time. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- We can represent the left-most recurrent perceptron in fig. 3 as a function of time:
 - at time step 0 (the first time step), the perceptron receives input at time 0
 - at time step t (where $t > 0$), the perceptron receives not only input at time t , $x(t)$, but also its output at time $t - 1$, $y(t-1)$
- This representation is called *Unrolling the Network Through Time*.

Recurrent Layer

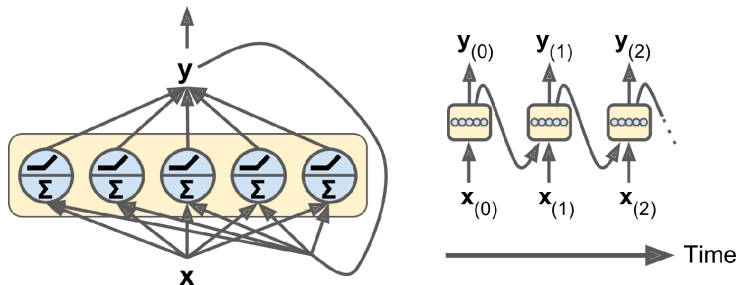


Figure 4: A layer of recurrent perceptrons unrolled through time. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- We can create a layer of recurrent perceptrons and represent it as a function of time:
 - at time step 0 (the first time step), each perceptron on the layer receives input at time 0, $x_{(0)}$
 - at time step t (where $t > 0$), each perceptron on the layer receives:
 - input at time t , $x_{(t)}$
 - output at time $t - 1$, $y_{(t-1)}$

Output of a Recurrent Layer

- The output of a recurrent layer at time step t , $\mathbf{y}_{(t)}$, can be written as

$$\mathbf{y}_{(t)} = \mathbf{f} \left(\mathbf{W}_x \mathbf{x}_{(t)}^\top + \mathbf{W}_y \mathbf{y}_{(t-1)}^\top + \mathbf{b} \right). \quad (1)$$

Here:

- $\mathbf{f}$ is the activation function (e.g., ReLU) of the layer
- $\mathbf{x}_{(t)}$ is the input at time t
- $\mathbf{W}_x$ is the connecting weight between the perceptrons on the layer and $\mathbf{x}_{(t)}$
- $\mathbf{y}_{(t-1)}$ is the output of the layer at time $t - 1$
- $\mathbf{W}_y$ is the connecting weight between the perceptrons on the layer and $\mathbf{y}_{(t-1)}$
- $\mathbf{b}$ is the bias of the perceptrons on the layer
- The recursive relationship in eq. (1) shows that $\mathbf{y}_{(t)}$ is a function of the input across all the previous time steps, $\mathbf{x}_{(0)}, \mathbf{x}_{(1)}, \dots, \mathbf{x}_{(t-1)}$.

Memory Cell

- As mentioned earlier, the output of a recurrent perceptron at a time step is a function of the input across all the previous time steps.
- That is, a recurrent perceptron has a form of memory that can preserve some state (more on this later) across time steps.
- Indeed, a recurrent perceptron is one kind of memory cell that can only learn short patterns (typically about 10 steps long).
- This kind of cell is also called *Short-Term Memory Cell*.
- There are more complex memory cells that allow longer patterns (typically about 10 times longer, more on this later).
- This kind of cell is also called *Long-Term Memory Cell*.

Output and Hidden State

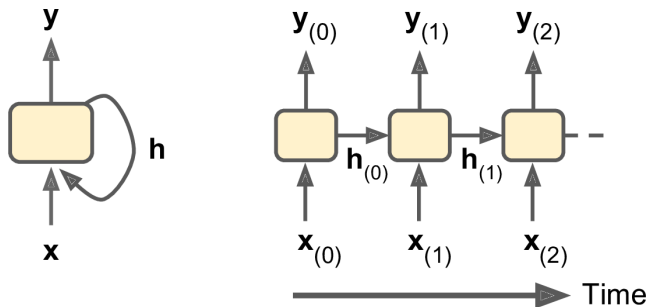


Figure 5: Output and hidden state of a memory cell. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- A memory cell's state at time step t , $\mathbf{h}_{(t)}$, is a function of the input at t , $\mathbf{x}_t$, and its state at $t - 1$, $\mathbf{h}_{(t-1)}$.
- A memory cell's output at time step t , $\mathbf{y}_{(t)}$, is also a function of the input at t , $\mathbf{x}_t$, and its output at $t - 1$, $\mathbf{y}_{(t-1)}$.
- While $\mathbf{h}_{(t)}$ and $\mathbf{y}_{(t)}$ could be the same in simple cells (left panel in fig. 5), they might be different in more complex cells (right panel in the figure).

Input and Output Sequences

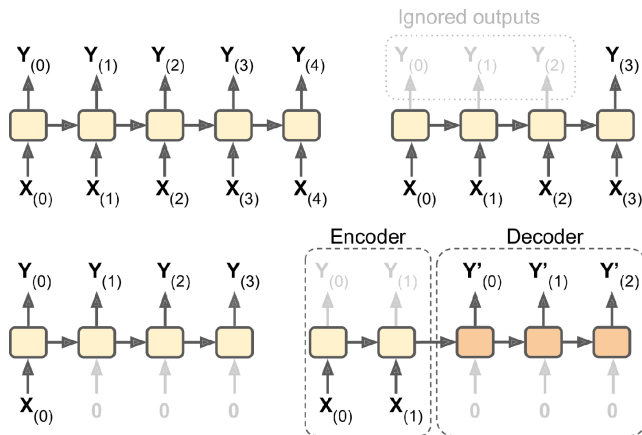


Figure 6: Four types of input and output sequences of RNNs. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

Input and Output Sequences

- *Sequence-to-Sequence Network* (top-left panel in fig. 6):
 - RNNs that take as input a sequence and output a sequence
 - e.g., given the input (a sequence), predict the output (also a sequence) that is some time steps later than the input
- *Sequence-to-Vector Network* (top-right panel in fig. 6):
 - RNNs that take as input a sequence and output a vector
 - e.g., given the book review (a sequence), predict the book rating (a vector)
- *Vector-to-Sequence Network* (bottom-left panel in fig. 6):
 - RNNs that take as input a vector and output a sequence
 - e.g., given an image (a vector), predict its caption (a sequence)
- *Encoder-Decoder Network* (bottom-right panel in fig. 6):
 - RNNs that begin with a sequence-to-vector network, named *Encoder*, and end with a vector-to-sequence network, named *Decoder*
 - e.g., given a sentence in one language (a sequence), encode it into a latent representation (a vector), which is then decoded into a sentence in another language (a sequence)
 - sequence-to-sequence network may not work here as we may have to see the whole sentence to know its meaning

Deep RNNs

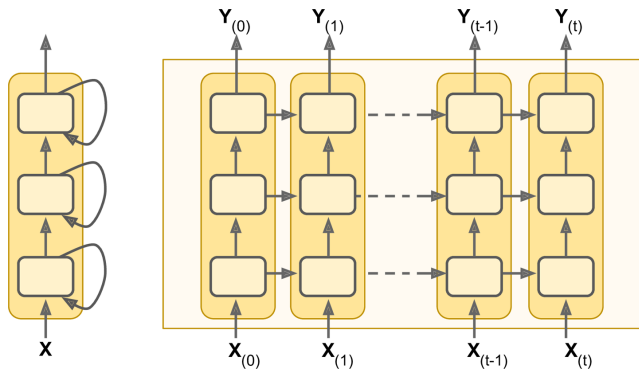


Figure 7: A deep RNN with three layers. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- Previously we have been focusing on RNNs that have only one hidden layer.
- Similar to DNNs, RNNs can also have many hidden layers, making them *Deep RNNs*.
- Fig. 7 shows a deep RNN with three layers.

Further Reading

- See HML: Chap 15 for a high-level description of a special backpropagation particularly designed for RNNs, named *Backpropagation Through Time* (BPTT).
- See NND: Chap 14 for:
 - a detailed discussion of BPTT
 - a detailed discussion of another backpropagation, also particularly designed for RNNs, named *Real-Time Recurrent Learning* (RTRL)
 - the pros and cons of BPTT and RTRL

The Short-Term Memory Problem

- After we pass data to RNNs, the data will go through a series of transformations (e.g., when calculating the net input and activation on each layer).
- Such transformations can also repeat themselves since we feed the output of RNNs back to the input layer.
- As a result, when RNNs receive the later elements of a long sequence, the fed back output may be very different from the earlier elements of the long sequence.
- In other words, we can think of RNNs as *Dory the fish* who, when translating the last word in a long sentence, already forgets about the first word in the sentence.
- This is called the *Short-Term Memory Problem*.

Long-Term Memory Cell

- To address the short-term memory problem of short-term memory cells, many cells with long-term memory have been proposed.
- Here we will introduce two popular long-term memory cells:
 - *Long Short-Term Memory* (LSTM) cell [Hochreiter and Schmidhuber, 1997]
 - *Gated Recurrent Unit* (GRU) cell [Cho et al., 2014]
- Compared to short-term memory cells, LSTM and GRU usually perform much better:
 - training LSTM and GRU could converge faster
 - LSTM and GRU could detect long-term dependencies in the data

LSTM Cell, As a Blackbox

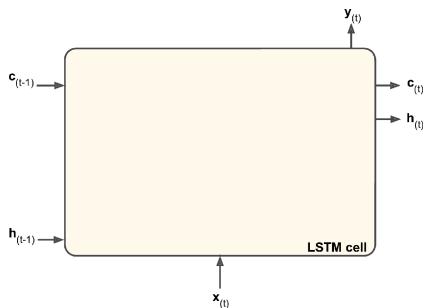


Figure 8: LSTM cell, as a blackbox. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- If we were to treat LSTM as a blackbox and only focus on its input and output, then the only difference between LSTM and short-term memory cell would be:
 - short-term memory cell has input $x_{(t)}$, output $y_{(t)}$ and short-term states $h_{(t-1)}$ and $h_{(t)}$
 - LSTM has extra long-term states $c_{(t-1)}$ and $c_{(t)}$
- The long-term states are the reason why LSTM works better on long sequences.

LSTM Cell: Architecture

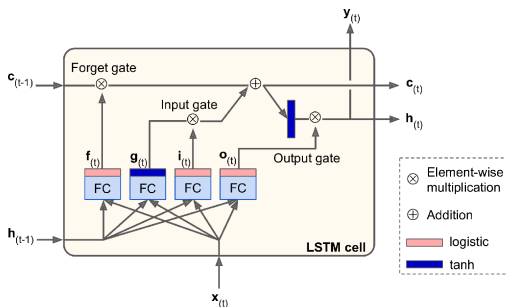


Figure 9: The architecture of LSTM cell. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- There are four kinds of fully connected layers in LSTM, which all take as input $h_{(t-1)}$ and $x_{(t)}$.
- Concretely:
 - the *Main Layer* outputs the activation ($\tanh$), $g_{(t)}$
 - the *Forget Gate Controller* outputs the activation (sigmoid), $f_{(t)}$
 - the *Input Gate Controller* outputs the activation (sigmoid), $i_{(t)}$
 - The *Output Gate Controller* outputs the activation (sigmoid), $o_{(t)}$

LSTM Cell: Architecture

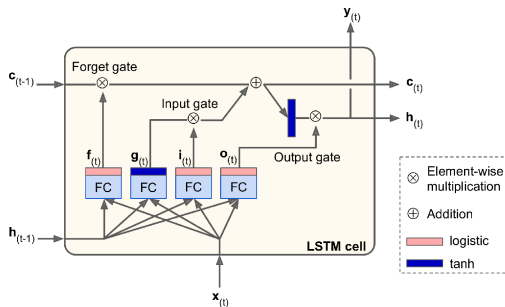


Figure 9: The architecture of LSTM cell. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- There are three major steps in LSTM:
 - recognize important input, with the *Input Gate* (taking as input $g_{(t)}$ and $i_{(t)}$)
 - store the important input in the long-term state for as long as it is needed, with the *Forget Gate* (taking as input $c_{(t-1)}$ and $f_{(t)}$)
 - extract the important input whenever it is needed, with the *Output Gate* (taking as input $\tanh(c_{(t)})$ and $o_{(t)}$)

LSTM Cell: Math

- The output of the input gate controller at time step t , $\mathbf{i}_{(t)}$, is

$$\mathbf{i}_{(t)} = \text{sigmoid} \left(\mathbf{W}_{xi} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hi} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_i \right). \quad (2)$$

- The output of the forget gate controller at time step t , $\mathbf{f}_{(t)}$, is

$$\mathbf{f}_{(t)} = \text{sigmoid} \left(\mathbf{W}_{xf} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hf} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_f \right). \quad (3)$$

- The output of the output gate controller at time step t , $\mathbf{o}_{(t)}$, is

$$\mathbf{o}_{(t)} = \text{sigmoid} \left(\mathbf{W}_{xo} \mathbf{x}_{(t)}^\top + \mathbf{W}_{ho} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_o \right). \quad (4)$$

- The output of the main layer at time step t , $\mathbf{g}_{(t)}$, is

$$\mathbf{g}_{(t)} = \tanh \left(\mathbf{W}_{xg} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hg} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_g \right). \quad (5)$$

- The long-term state at time step t , $\mathbf{c}_{(t)}$, is

$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}. \quad (6)$$

- The short-term state and output at time step t , $\mathbf{h}_{(t)}$ and $\mathbf{y}_{(t)}$, are

$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)}). \quad (7)$$

LSTM Cell: Math

- In eqs. (2) to (5):

- $\mathbf{W}_{xi}$, $\mathbf{W}_{xf}$, $\mathbf{W}_{xo}$ and $\mathbf{W}_{xg}$ are the connecting weights between the input at time step t , $\mathbf{x}_{(t)}$, and the input gate controller, forget gate controller, output gate controller and the main layer
- $\mathbf{W}_{hi}$, $\mathbf{W}_{hf}$, $\mathbf{W}_{ho}$ and $\mathbf{W}_{hg}$ are the connecting weights between the short-term state at time step $t - 1$, $\mathbf{h}_{(t-1)}$, and the input gate controller, forget gate controller, output gate controller and the main layer
- $\mathbf{b}_i$, $\mathbf{b}_f$, $\mathbf{b}_o$ and $\mathbf{b}_g$ are the biases of the input gate controller, forget gate controller, output gate controller and the main layer

- In eq. (6):

- $\mathbf{f}_{(t)}$, $\mathbf{i}_{(t)}$ and $\mathbf{g}_{(t)}$ are the output of the forget gate controller, input gate controller and the main layer at time step t
- $\mathbf{c}_{(t-1)}$ is the long-term state at time step $t - 1$

- In eq. (7):

- $\mathbf{o}_{(t)}$ is the output of the output gate controller at time step t
- $\mathbf{c}_{(t)}$ is the long-term state at time step t

Peephole Connections

- In regular LSTM, the input/forget/output gate controller only receives the input at time step t , $\mathbf{x}_{(t)}$, and the short-term state at time step $t - 1$, $\mathbf{h}_{(t-1)}$.
- An extension of LSTM with extra connections called *Peephole Connections* was proposed in [Gers and Schmidhuber, 2000]:
 - the long-term state at time step $t - 1$, $\mathbf{c}_{(t-1)}$, is added as an input to the input and forget gate controller
 - the long-term state at time step t , $\mathbf{c}_{(t)}$, is added as an input to the output gate controller

GRU Cell, As a Blackbox

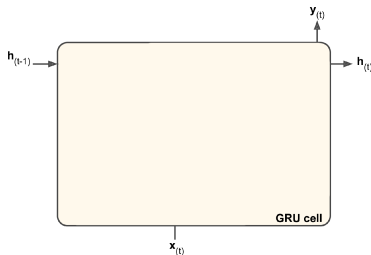


Figure 10: GRU cell, as a blackbox. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- If we were to treat GRU as a blackbox and only focus on its input and output, then there would be no difference between GRU and short-term memory cell:
 - both cells have input $\mathbf{x}_{(t)}$, output $\mathbf{y}_{(t)}$ and short-term states $\mathbf{h}_{(t-1)}$ and $\mathbf{h}_{(t)}$
- Note that LSTM has extra long-term states $\mathbf{c}_{(t-1)}$ and $\mathbf{c}_{(t)}$.

GRU Cell: Architecture

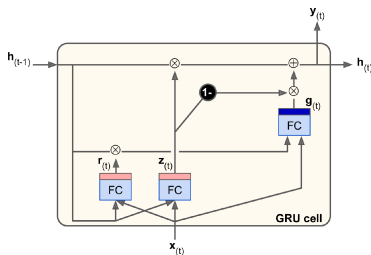


Figure 11: The architecture of GRU cell. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- There are three kinds of fully connected layers in GRU:
 - a gate controller (receiving $h_{(t-1)}$ and $x_{(t)}$) outputs the activation (sigmoid), $z_{(t)}$
 - a gate controller (receiving $h_{(t-1)}$ and $x_{(t)}$) outputs the activation (sigmoid), $r_{(t)}$
 - the *Main Layer* (receiving $r_{(t)}$ and $x_{(t)}$) outputs the activation (tanh), $g_{(t)}$

GRU Cell: Architecture

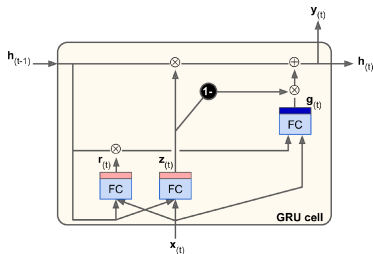


Figure 11: The architecture of GRU cell. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

- There are three major steps in GRU:
 - recognize important input (that should be used when producing the output), with gate controller that outputs $z_{(t)}$ and main layer that outputs $g_{(t)}$
 - store the important input (that should be used when producing the output) in the state for as long as it is needed, with gate controller that outputs $z_{(t)}$ (the 1- operation allows the controller to switch between its two functions)
 - recognize important input (that should be used when producing $g_{(t)}$), with gate controller that outputs $r_{(t)}$

GRU Cell: Math

- The output of a gate controller at time step t , $\mathbf{z}_{(t)}$, is

$$\mathbf{z}_{(t)} = \text{sigmoid} \left(\mathbf{W}_{xz} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hz} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_z \right). \quad (8)$$

- The output of a gate controller at time step t , $\mathbf{r}_{(t)}$, is

$$\mathbf{r}_{(t)} = \text{sigmoid} \left(\mathbf{W}_{xr} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hr} \mathbf{h}_{(t-1)}^\top + \mathbf{b}_r \right). \quad (9)$$

- The output of the main layer at time step t , $\mathbf{g}_{(t)}$, is

$$\mathbf{g}_{(t)} = \tanh \left(\mathbf{W}_{xg} \mathbf{x}_{(t)}^\top + \mathbf{W}_{hg} (\mathbf{r}_{(t)} \otimes \mathbf{h}_{(t-1)})^\top + \mathbf{b}_g \right). \quad (10)$$

- The short-term state and output at time step t , $\mathbf{h}_{(t)}$ and $\mathbf{y}_{(t)}$, are

$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{z}_{(t)} \otimes \mathbf{h}_{(t-1)} + (1 - \mathbf{z}_{(t)}) \otimes \mathbf{g}_{(t)}. \quad (11)$$

GRU Cell: Math

- In eqs. (8) to (10):
 - $\mathbf{W}_{xz}$, $\mathbf{W}_{xr}$ and $\mathbf{W}_{xg}$ are the connecting weights between the input at time step t , $\mathbf{x}_{(t)}$, and the two gate controllers and the main layer
 - $\mathbf{W}_{hz}$, $\mathbf{W}_{hr}$ and $\mathbf{W}_{hg}$ are the connecting weights between the short-term state at time step $t - 1$, $\mathbf{h}_{(t-1)}$, and the two gate controllers and the main layer
 - $\mathbf{b}_z$, $\mathbf{b}_r$ and $\mathbf{b}_g$ are the biases of the two gate controllers and the main layer
- In eq. (10):
 - $\mathbf{r}_{(t)}$ is the output of a gate controller at time step t
- In eq. (11):
 - $\mathbf{z}_{(t)}$ is the output of a gate controller at time step t
 - $\mathbf{g}_{(t)}$ is the output of the main layer at time step t

Encoder-Decoder based Neural Machine Translation

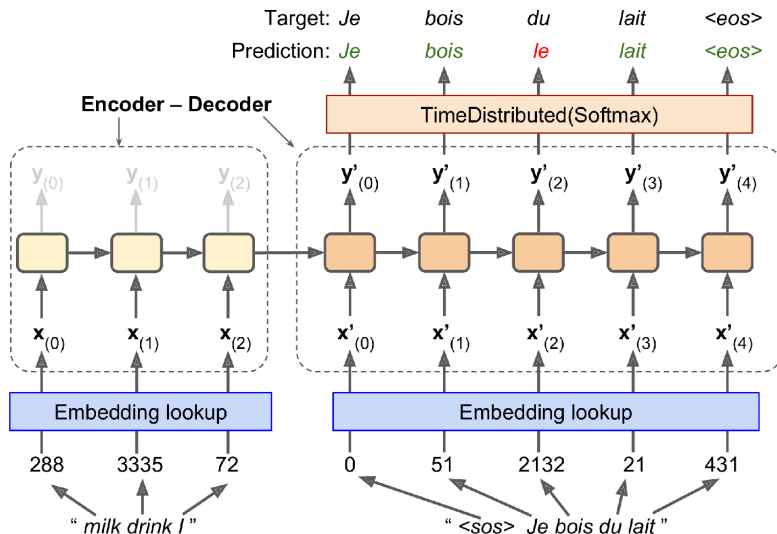


Figure 12: An encoder-decoder based neural machine translation model.

Encoder-Decoder based Neural Machine Translation

- Fig. 12 shows that, the encoder takes as input a reversed English sentence and outputs the hidden states (a latent representation of the input sentence).
- The input sentence was reversed because the decoder needs to first translate the last word in the reversed sentence (i.e., first word in the original sentence).
- The decoder takes as input the hidden states of the encoder and
 - the target French sentence shifted by one step (during training), where $\langle \text{sos} \rangle$ stands for *start of sentence*
 - the output of the decoder at the previous step, as shown in fig. 13 (during testing)
- The decoder outputs the translated French sentence where $\langle \text{eos} \rangle$ (as shown in fig. 12) stands for *end of sentence*.

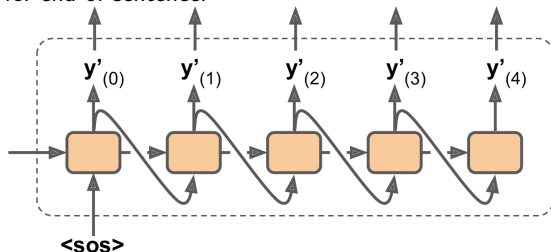


Figure 13: The input of decoder during testing.

The Idea of Bidirectional RNNs

- For all the RNNs discussed so far, their recurrent layers only look at the past inputs to predict the output:
 - “I have not had any meal today so I am ____.”
 - based on the words before the blank, *hungry* is more likely than *full*
- However, sometimes we may have to look at the future inputs to predict the output:
 - “I am ____ because I have not had any meal today.”
 - if we were to only consider the words before the blank, *hungry* and *full* would be equally likely (which is wrong)
 - if, instead, we can somehow also consider the words after the blank, *hungry* will be more likely than *full* (which is correct)
- The idea of *Bidirectional RNNs* is using both the past and future inputs (hence the name) for prediction.

The Architecture of Bidirectional RNNs

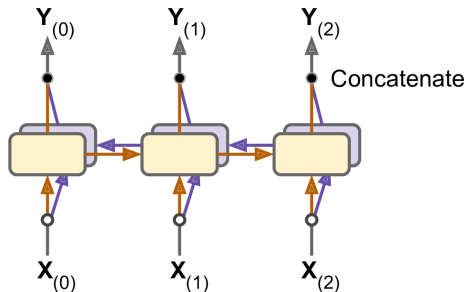


Figure 14: A bidirectional recurrent layer.

- The key building block in bidirectional RNNs is the *Bidirectional Recurrent Layer* (see fig. 14).
- A bidirectional recurrent layer comprises two recurrent layers:
 - one passes the input from left to right (to consider inputs before the blank)
 - one passes the input from right to left (to consider inputs after the blank)
- The output of the two recurrent layers will then be combined (typically concatenated).

The idea of Attention Mechanisms

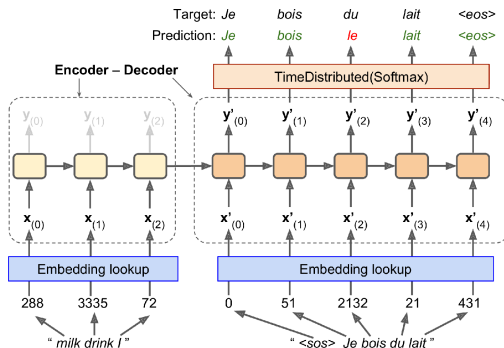


Figure 12: An encoder-decoder based neural machine translation model.

- As shown in fig. 12, in standard encoder-decoder based neural machine translation model, the path from "milk" to "lait" is fairly long.
- This long path could compromise the translation accuracy due to the short-term memory limitations of RNNs.
- The idea of the *Attention Mechanism* is that, at the time step where the decoder should output "lait", it will focus its attention on "milk" (so as to shorten the path between the two).

The Architecture of Attention Mechanisms

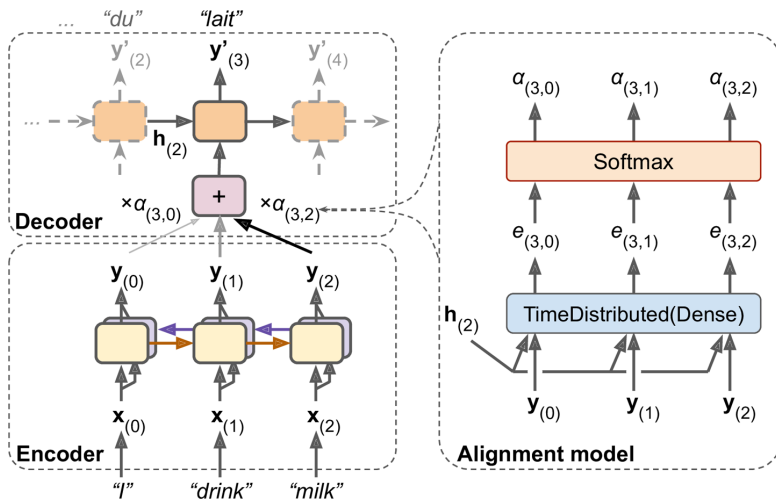


Figure 15: An encoder-decoder based neural machine translation model with attention.

The Architecture of Attention Mechanisms

- Unlike in standard encoder-decoder neural machine translation model where the decoder only receives the hidden state from the encoder, in attention mechanism the decoder at each time step also receives the output of the encoder across each time step.
- At each time step, the decoder calculates a weighted sum of the encoder outputs.
- This weighted sum determines which words the decoder should focus on at this time step.
- Concretely, weight $\alpha_{(t,i)}$ is the weight of the i^{th} encoder output at the t^{th} decoder time step.
- For example, in fig. 15:
 - $\alpha_{(3,0)}$ is the weight of the 0th encoder output at the 3rd decoder time step
 - $\alpha_{(3,1)}$ is the weight of the 1st encoder output at the 3rd decoder time step
 - $\alpha_{(3,2)}$ is the weight of the 2nd encoder output at the 3rd decoder time step
- If $\alpha_{(3,2)}$ is much larger than $\alpha_{(3,0)}$ and $\alpha_{(3,1)}$, the decoder will pay more attention to the 2nd encoder output, and in turn, the 2nd encoder input (word “milk”).

Bahdanau Attention

- The weights in fig. 15, $\alpha_{(t,i)}$, are calculated by a small neural network named *Attention Layer* (a.k.a., *Alignment Model*).
- As shown in fig. 15, an attention layer comprises:
 - a TimeDistributed Dense Layer
 - a Softmax Layer
- A *TimeDistributed Layer* can wrap any kind of layer (e.g., Dense) and apply it to each time step of its input sequence (so as to output a sequence).
- Concretely, for a decoder time step (e.g., 3), the time-distributed dense layer:
 - takes as input the encoder output and the decoder's previous hidden state (e.g., $\mathbf{h}_{(2)}$)
 - outputs a score (e.g., $e_{(3,2)}$) for each encoder output, measuring how well the encoder output aligns with the decoder's previous hidden state (the higher the score the better the alignment)
- For a decoder time step (e.g., 3), the softmax layer:
 - takes as input the output of the TimeDistributed layer
 - outputs a probability distribution (i.e., the weights $\alpha_{(t,i)}$)
- This particular attention mechanism is called *Bahdanau Attention* (a.k.a., *Concatenative Attention* or *Additive Attention*).

Other Attention Mechanisms

- Inspired by Bahdanau Attention, other attention mechanisms have also been proposed.
- Since the idea of the attention mechanism is measuring the similarity between an encoder output and the decoder's previous hidden state, a more straightforward way (compared to Bahdanau attention) for doing so is calculating the dot product of the two vectors, as:
 - dot product is a fairly good measurement for similarity
 - dot product can be calculated in parallel (so that it is much faster to compute)
- An extension of the above dot product:
 - 1 first uses a time-distributed dense layer to conduct a linear transformation (i.e., weighted sum) for the encoder output
 - 2 then calculates the dot product of the transformed encoder output and decoder's previous hidden state
- This particular mechanism is called *Luong Attention* (a.k.a., *Multiplicative Attention*).
- Other differences between Luong attention and Bahdanau attention include:
 - Luong attention uses decoder's current hidden state, $\mathbf{h}_{(t)}$, rather than the previous hidden state, $\mathbf{h}_{(t-1)}$, to calculate the similarity score, $e_{(t,i)}$
 - Luong attention uses the output of the attention mechanism, $\tilde{\mathbf{h}}_{(t)}$, to calculate the decode predictions, rather than decoder's current hidden state

The Math of Attention Mechanisms

- Concretely, the similarity score between encoder's i^{th} output ($\mathbf{y}_{(i)}$) and decoder's t^{th} hidden state ($\mathbf{h}_{(t)}$), $e_{(t,i)}$, is calculated as:

$$e_{(t,i)} = \begin{cases} \mathbf{h}_{(t)}^{\top} \mathbf{y}_{(i)}, & \text{dot} \\ \mathbf{h}_{(t)}^{\top} \mathbf{W} \mathbf{y}_{(i)}, & \text{general} \\ \mathbf{v}^{\top} \tanh(\mathbf{W} [\mathbf{h}_{(t)}, \mathbf{y}_{(i)}]) . & \text{concat} \end{cases} \quad (12)$$

Here:

- $\mathbf{W}$ is the weight matrix
- $\mathbf{v}^{\top}$ is a scaling parameter vector
- The weight between encoder's i^{th} output ($\mathbf{y}_{(i)}$) and decoder's t^{th} hidden state ($\mathbf{h}_{(t)}$), $\alpha_{(t,i)}$, is calculated as:

$$\alpha_{(t,i)} = \frac{\exp(e_{(t,i)})}{\sum_i \exp(e_{(t,i)})} . \quad (13)$$

- The output of the attention mechanism at time step t , $\tilde{\mathbf{h}}_{(t)}$, is calculated as:

$$\tilde{\mathbf{h}}_{(t)} = \sum_i \alpha_{(t,i)} \mathbf{y}_{(i)} . \quad (14)$$

Attention is All You Need

- A key improvement over the attention mechanism was proposed in a groundbreaking paper [Vaswani et al., 2017], which suggests “Attention is All You Need”.
- The corresponding model, named *Transformer*, significantly improved the state-of-the-art in NMT.
- Similar to previous models for NMT, transformer also comprises an encoder and decoder.
- Unlike previous models for NMT, the key building blocks in transformer are not recurrent or convolutional layers, but only attention mechanisms.
- Fig. 16 shows the architecture of transformer.
- [Here](#) is a very nice tutorial (from Google) for building transformer.
- [Here](#) is a much more detailed tutorial (from Huggingface) for building transformer and its extensions.

The Architecture of Transformer

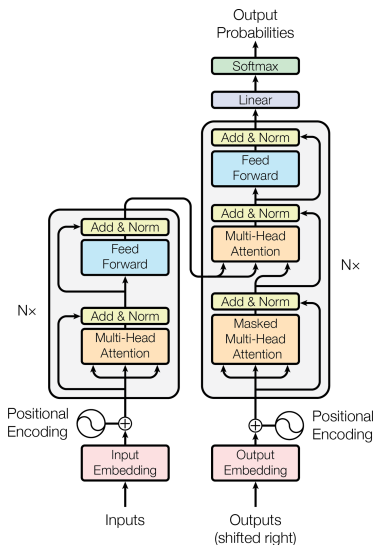


Figure 16: The architecture of transformer.

Encoder

- The encoder in transformer is shown in the lefthand part of fig. 16.
- Concretely, the encoder:
 - takes as input a batch of sentences represented as sequences of word IDs produced by an embedding layer (with input shape [batch size, max input sentence length])
 - encodes each word into a 512-dimensional representation (with output shape [batch size, max input sentence length, 512])
- Particularly, the top part of the encoder is stacked N times (with $N = 6$ in the paper).

Decoder

- The decoder in transformer is shown in the righthand part of fig. 16.
- Concretely, the decoder:
 - (during training) takes as input the target sentence represented as a sequence of wordIDs produced by an embedding layer, shifted one time step to the right (with a *sos* token inserted in the beginning)
 - (during testing) takes as input the previous output words (with a *sos* token inserted in the beginning)
 - takes as input the output of the encoder
 - outputs the probability distribution over the possible next word (with output shape [batch size, max output sentence length, vocabulary length]), at each time step
- Similar to the encoder, the top part of the decoder is also stacked N times (with $N = 6$ in the paper).

Positional Encodings

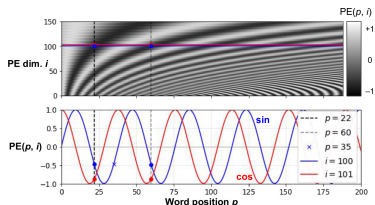


Figure 17: An example of positional encoding.

- Since transformer does not use RNN layers, it does not know the ordering of the words in a sentence, which can be crucial in NMT.
- To address this problem, transformer uses *Positional Encoding*, which is a dense vector that encodes the position of a word in a sentence.
- Formally, entry (p, i) in the positional matrix, $pe_{p,i}$, is

$$pe_{p,i} = \begin{cases} \sin\left(\frac{p}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even,} \\ \cos\left(\frac{p}{10000^{\frac{i-1}{d}}}\right) & \text{if } i \text{ is odd,} \end{cases} \quad (15)$$

where $pe_{p,i}$ is the i^{th} component of the encoding for the p^{th} word in the sentence.

- Fig. 17 shows an example of positional encoding.

Multi-Head Attention

- One key building block in transformer is the *Multi-Head Attention Layer* in the encoder and decoder.
- Concretely, the multi-head attention layer in the encoder encodes each word's relationship with every other word in the same sentence, paying more attention to the most relevant ones:
 - considering the input sentence "I gotta hit the road"
 - the output of this layer for word "hit" will pay more attention to "road" than other words in the sentence
- This attention mechanism is called *Self-Attention* (as the sentence pays attention to itself).
- The multi-head attention layer in the decoder pays attention to the words in the input sentence that are about to be translated:
 - again, considering the input sentence "I gotta hit the road"
 - the output of this layer for word "hit" will pay more attention to "hit" than other words in the sentence

Masked Multi-Head Attention

- Besides the multi-head attention layer, the other key building block in transformer is the *Masked Multi-Head Attention Layer*.
- Unlike the multi-head attention layer which resides in both the encoder and decoder, the masked multi-head attention layer only resides in the decoder.
- Concretely, the masked multi-head attention layer in the decoder is both similar to and different from the multi-head attention layer in the encoder:
 - similarity: both layers rely on the self-attention mechanism to pay attention to the words in their input sentence
 - difference:
 - for each word in the input sentence of the encoder, the multi-head attention layer pays attention to every word in the sentence
 - for each word in the input sentence of the decoder, the masked multi-head attention layer only pays attention to words before it (more on this later)

Scaled Dot-Product Attention Layer

- Both the multi-head attention layer and the masked multi-head attention layer are based on the *Scaled Dot-Product Attention Layer*.
- The idea of the scaled dot-product attention layer is fairly similar to a differentiable dictionary lookup.
- Again, considering the input sentence “I gotta hit the road”.
- The representation of the input sentence output by the encoder can be thought of as a dictionary: {key 1: I, key 2: gotta, key 3: hit, key 4: the, key 5: road}.
- When the decoder is about to translate a word, say “hit”, the decoder proposes a *Query*, which ideally should match the key of “hit”, key 3.
- Since both key 3 and the query could be float, they may not perfectly match.
- A work around is:
 - calculating a similarity score (e.g., dot product) between the query and each key in the dictionary: {score 1, score 2, score 3, score 4, score 5}
 - using softmax to transform these scores into weights that add up to 1 (i.e., a probability distribution): {weight 1, weight 2, weight 3, weight 4, weight 5}
 - calculating a weighted sum of the words in the dictionary: {weight 1 $\times$ “I”, weight 2 $\times$ “gotta”, weight 3 $\times$ “hit”, weight 4 $\times$ “the”, weight 5 $\times$ “road”}
- The idea is, if weight 3 is close to 1, the weighted sum will be close to “hit”.

Scaled Dot-Product Attention Layer

- Formally, scaled dot-product attention is

$$\text{Attention}(\mathbf{Q}, \mathbf{k}, \mathbf{v}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{keys}}}}\right)\mathbf{V}, \quad \text{where :} \quad (16)$$

- $\mathbf{Q}$ is a $n_{\text{queries}} \times d_{\text{keys}}$ matrix, where n_{queries} is the number of queries and d_{keys} the dimension of each query and each key
- $\mathbf{K}$ is a $n_{\text{keys}} \times d_{\text{keys}}$ matrix, where n_{keys} is the number of keys and values, and d_{keys} the dimension of each query and each key
- $\mathbf{V}$ is a $n_{\text{keys}} \times d_{\text{values}}$ matrix, where n_{keys} is the number of keys and values, and d_{values} the dimension of each value
- $\mathbf{Q}\mathbf{K}^\top$ is a $n_{\text{queries}} \times n_{\text{keys}}$ matrix, where each entry is a similarity score (e.g., dot product) for each query-key pair
- $\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{keys}}}}\right)$ is a $n_{\text{queries}} \times n_{\text{keys}}$ matrix, where for each row the sum across the columns is 1
- $\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{keys}}}}\right)\mathbf{V}$ is a $n_{\text{queries}} \times d_{\text{values}}$ matrix, where each row is the query result (weighted sum of the values)
- the scaling factor, $\sqrt{d_{\text{keys}}}$, scales down the similarity score so that it is less likely to saturate the softmax function, which would lead to tiny gradients
- in the masked multi-head attention layer, we can mask out key-value by adding a very large negative value to the corresponding value (so that the probability produced by the softmax function will be close to zero)

Scaled Dot-Product Attention Layer

- In the multi-head attention layer in the encoder:
 - we apply eq. (16) to each input sentence in the batch, where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are all words in the sentence
 - as a result, each word in a sentence will be compared to every word in the same sentence (including the word itself)
- In the multi-head attention layer in the decoder:
 - $\mathbf{K}$ and $\mathbf{V}$ are word encodings produced by the encoder
 - $\mathbf{Q}$ are word encodings produced by the decoder
- In the masked multi-head attention layer in the decoder:
 - we apply eq. (16) to each input sentence in the batch (i.e., the target during training or the decoder's output in previous time step during testing), where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are all words in the sentence
 - we use a mask so that we will not compare a word to those behind it (as discussed earlier)

The Architecture of Multi-Head Attention

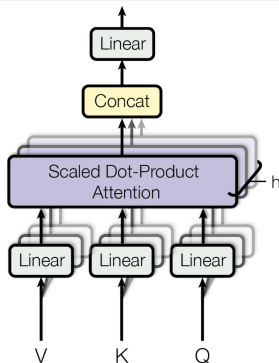


Figure 18: The architecture of multi-head attention.

- With the knowledge of scaled dot-product attention layer, we can finally introduce the architecture of multi-head attention.
- As shown in fig. 18, multi-head attention is simply a stack of scaled dot-product attention layers, which:
 - take as input a linear transformation of the values (V), keys (K) and queries (Q)
 - output the weighted sum of the values in eq. (16), which will first be concatenated and then be passed to a linear transformation

The Idea of Multi-Head Attention

- A natural question is, what is the benefit of using multi-head attention (which comprises a stack of scaled dot-product attention layer)?
- Considering the sentence, “He failed the test.”, where the word “failed” has different characteristics:
 - it is a verb
 - it is in the past tense
 - it has a negative connotation
- Using multi-head attention:
 - we can apply different linear transformations of the values, keys and queries, which can project the word representations onto different subspaces, each focusing on one kind of the word's characteristics
 - we can apply different scaled dot-product attention layers, each focusing on one kind of lookup phase
 - we can concatenate the output of the scaled dot-product attention layers and project them back to the original space

Transformer: Other Components

- Besides the attentional layers discussed earlier, transformer also comprises other components:
 - two embedding layers
 - $5 \times N$ skip connections (see [p3_c2_s3_convolutional_neural_networks](#)), each of them followed by a normalization layer
 - $2 \times N$ FNNs each of which comprises two dense layers:
 - the first dense layer uses ReLU as the activation function
 - the second dense layer uses Identity as the activation function
 - an output (dense) layer using Softmax as the activation function
- All of the above components are time-distributed, so that each word is treated independently of all the others.

Transfer Learning

- Similar to transfer learning with CNNs:
 - when there are state-of-the-art RNNs pretrained on similar data, it is recommended to reuse and tweak the pretrained RNNs to make them suitable for our data
 - transfer learning will not only speed up building, training and fine-tuning RNNs considerably, but also require significantly less training data
 - it turns out that transfer learning also works particularly well in NLP, since the lower layers of a pretrained RNNs will usually capture simple features that are common in many data (hence can be reused)

State-Of-The-Art Pretrained RNNs

- See state-of-the-art pretrained RNNs in [TensorFlow Hub](#).

Building RNNs with Pretrained RNNs

- Here we can simply follow the good practice (for building DNNs with pretrained DNNs) discussed previously (also shown below).



Good practice

- To build a DNN with pretrained model, we should:
 - ① reuse the lower layers of the pretrained model as the base
 - ② add extra layers (that work for our data) on top of the base
- The more similar our data is to the data where the model was pretrained, the more lower layers of the pretrained model we should reuse as the base.
- It is even possible to reuse all the hidden layers of a pretrained model, when the data are similar enough.
- We should resize our data so that the number of features in the resized data is the same as the number of perceptrons on the input layer of the pretrained model.

Training RNN with Pretrained RNN

- Here we can simply follow the good practice (for training DNNs with pretrained DNNs) discussed previously (also shown below).



Good practice

- To train a DNN with pretrained model, we should:
 - ① freeze all the reused layers of the pretrained DNN (i.e., make their weights non-trainable so that backpropagation will not change them) then train the DNN
 - ② unfreeze one or two top hidden layers of the pretrained DNN (the more training data we have the more top hidden layers we can unfreeze) and reduce the learning rate when doing so (thus the fine-tuned weights on the lower layers will not change significantly)
- If the above steps do not produce an accurate DNN:
 - if we do not have sufficient data, we can drop the top hidden layers and repeat the above steps
 - otherwise, we can replace (rather than drop) the top hidden layers or even add more hidden layers

Designing and Training DNN with Pretrained DNN

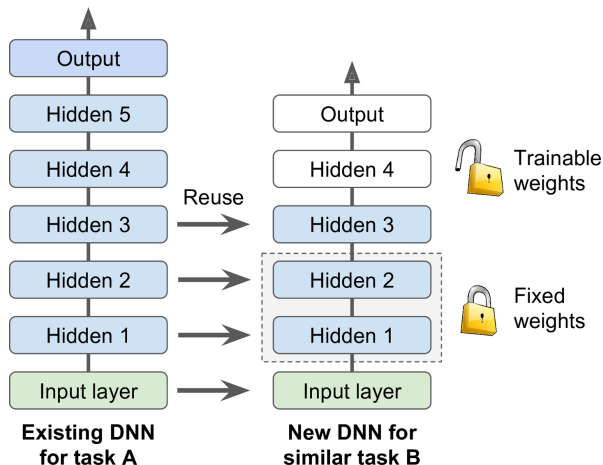


Figure 19: Building and training DNN with pretrained model. Picture courtesy of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*.

Building RNN with Pretrained RNN: Code Example

- See /p3_c2_s4_recurrent_neural_networks/case_study:
 - 1 cell 16

Freezing the Pretrained Layers: Code Example

- See /p3_c2_s4_recurrent_neural_networks/case_study:
 - 1 cell 17

Unfreezing the Pretrained Layers: Code Example

- See /p3_c2_s4_recurrent_neural_networks/case_study:
 - 1 cell 23

Bibliography I



Cho, K., Merriënboer, B. V., Bahdanau, B., and Bengio, Y. (2014).
On the Properties of Neural Machine Translation: Encoder-Decoder Approaches.
arXiv preprint arXiv:1409.1259.



Gers, F. A. and Schmidhuber, J. (2000).
Recurrent Nets that Time and Count.
In Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium, volume 3, pages 189–194. IEEE.



Hochreiter, S. and Schmidhuber, J. (1997).
Long Short-Term Memory.
Neural computation, 9(8):1735–1780.



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017).
Attention is All You Need.
In Advances in neural information processing systems, pages 5998–6008.